

JavaScript: Requisições, APIs e Assincronicidade

Conceitos, Prática e Projeto Blog

```
fetch('/api/v1/knowledge').then(res => res.json());
```

Mapa dos assuntos

01 Introdução a requisições

02 Síncrono e Assíncrono

03 Como a Web funciona (Req e Resp)

04 Como uma API funciona (JSON)

05 O que é um Callback?

06 Fazendo uma requisição na prática

07 Ver requisições no DevTools

08 Entendendo Promise

09 Gerenciando erros na Promise

10 Tipos de status de uma requisição

11 Métodos de envio (GET, POST, etc)

12 Promises com async/await

13 Projeto: Blog do JSONPlaceholder 1

14 Projeto: Blog do JSONPlaceholder 2

Síncrono e Assíncrono

Síncrono (Bloqueante)

As tarefas são executadas uma após a outra. A próxima tarefa só começa quando a anterior termina.

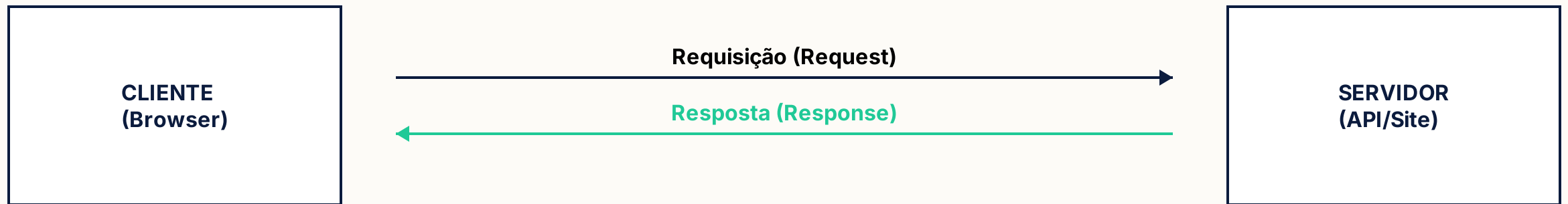
```
console.log("Passo 1");  
console.log("Passo 2");  
console.log("Passo 3");
```

Assíncrono (Não-bloqueante)

Permite que tarefas demoradas (como requisições) rodem em paralelo sem travar a execução do script principal.

```
console.log("Início");  
setTimeout(() => {  
  console.log("Processado");  
}, 2000);  
console.log("Fim");
```

Como a Web funciona: Req e Resp



Headers (Cabeçalhos)

Metadados da comunicação: tipo de conteúdo, autenticação, cookies, etc.

Body (Corpo)

Os dados reais sendo enviados ou recebidos (geralmente em formato JSON).

Como uma API funciona

Application Programming Interface

Uma API é uma ponte que permite que dois sistemas conversem. No desenvolvimento web, usamos APIs para buscar dados de um servidor e exibí-los no navegador.

JSON (JavaScript Object Notation): É o formato padrão de troca de dados. É leve, fácil de ler para humanos e fácil de processar para máquinas.

Exemplo de JSON

```
{  
  "id": 1,  
  "titulo": "Minha Postagem",  
  "autor": "Manus",  
  "tags": ["js", "api"]  
}
```

O que é um Callback?

Funções como Argumentos

Um callback é uma função que é passada como argumento para outra função, para ser executada posteriormente.

É a forma mais básica de lidar com assincronicidade: "Quando você terminar essa tarefa demorada, execute esta função aqui".

```
function exibir(resultado) {  
  console.log("Resultado: " + resultado);  
}  
  
function somar(a, b, callback) {  
  let res = a + b;  
  callback(res);  
}  
  
somar(5, 10, exibir);
```

Fetch e Promises na prática

A API Fetch

O `fetch()` é o método moderno para fazer requisições HTTP. Ele retorna uma **Promise**.

Promise: Um objeto que representa o sucesso ou a falha eventual de uma operação assíncrona.

Usamos o `.then()` para lidar com a resposta quando ela chega.

```
fetch('https://api.exemplo.com/posts')
  .then(response => {
    return response.json();
  })
  .then(json => {
    console.log(json);
  });
```

Observando requisições no DevTools

A Aba Network (Rede)

Pressione F12 e vá em "Network". Recarregue a página para ver todas as requisições que o navegador faz.

Name	Status	Type	Size	Time
posts	200	fetch	1.2kB	45ms
users	200	fetch	0.8kB	32ms
logo.png	200	png	15kB	12ms

O que analisar?

Clique na requisição para ver os **Headers**, o **Payload** (dados enviados) e a **Preview/Response** (dados recebidos).

Erros e Status HTTP

Status	Significado
200 / 201	Sucesso / Criado
301 / 302	Redirecionamento
400 / 401	Bad Request / Não Autorizado
404	Não Encontrado
500	Erro Interno do Servidor

```
fetch('...')
  .then(res => res.json())
  .catch(error => {
    console.error("Erro:", error);
  })
  .finally(() => {
    console.log("Fim da req");
  });
```

Métodos HTTP e envio de dados

Buscar dados

GET

Enviar novos dados

POST

Atualizar dados

PUT

Remover dados

DELETE

```
fetch('... ', {  
  method: 'POST',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  body: JSON.stringify({  
    title: 'Novo Post',  
    body: 'Conteúdo ... '  
  })  
});
```

Promises com async/await

Sintaxe com .then()

```
function carregar() {  
  fetch('...')  
    .then(res => res.json())  
    .then(json => {  
      console.log(json);  
    });  
}
```

Sintaxe com async/await

```
async function carregar() {  
  const res = await fetch('...');  
  const json = await res.json();  
  console.log(json);  
}
```

Projeto: Blog do JSONPlaceholder

Fase 1: Leitura

Consumir o endpoint `/posts` para listar todas as postagens na tela usando `fetch` e manipulação de DOM.

Fase 2: Inserção

Criar um formulário que captura título e corpo, enviando uma requisição **POST** para simular a criação de um post.

Ferramentas

- **API:** JSONPlaceholder (Fake API)
- **JS:** Async/Await, Fetch, DOM
- **CSS:** Layout responsivo para o blog

PRÁTICA GUIADA:
Da requisição à interface.